

SecGate — Team Cloud & Infra Budget Governance Playbook

SOP / Agent Playbook · Owner: SecGate Platform Security · Audience: AI agents approving cloud/infra deployments · Version 1.0

1. Purpose

This playbook defines how SecGate evaluates **team cloud and infrastructure spend** before approving AI-initiated deployments (GPU clusters, managed compute, long-running services). Agents **must** look up the requesting team's monthly budget and current spend, then approve or reject the proposal against remaining capacity. Budget data is exposed through **Nexla MCP tools** so guardians can call it at decision time.

2. Authoritative data model

Budget records live in a governed table (or ToolSet-backed dataset) with exactly these fields:

Field	Type	Description	Example
team	string	Team slug (lowercase, hyphenated)	platform-eng
monthly_budget_usd	number	Hard monthly cloud/infra budget (USD)	500
spent_usd	number	Month-to-date committed/spent (USD)	47

Canonical demo row

```
{ "team": "platform-eng", "monthly_budget_usd": 500, "spent_usd": 47 }
```

Derived: remaining_usd = monthly_budget_usd – spent_usd (example: 500 – 47 = **453**). Compute at call time; do not store separately.

3. Required MCP tools (infer from this SOP)

Nexla MCP Studio should generate a ToolSet exposing at least the tools below. Names and schemas are intentional so SecGate Guardian can bind without custom adapters.

3.1 get_team_budget (required)

Purpose: Return one team's monthly budget and current spend. This is the primary tool SecGate Guardian calls before approve/reject.

Argument	Type	Required	Description
team	string	no	Team slug. Default: platform-eng

Output (JSON object) — all fields required:

Field	Type	Notes
team	string	Echo of resolved team slug
monthly_budget_usd	number	Monthly hard cap in USD
spent_usd	number	Month-to-date spend in USD

Example response

```
{ "team": "platform-eng", "monthly_budget_usd": 500, "spent_usd": 47 }
```

3.2 list_team_budgets (recommended)

Purpose: List all team budget rows. **Input:** none. **Output:** JSON array of objects with the same three fields as §2.

```
[ { "team": "platform-eng", "monthly_budget_usd": 500, "spent_usd": 47 } ]
```

4. Deployment approval procedure

When an agent proposes a new infra deployment with projected monthly cost projected_monthly_usd:

1. **Identify team** — use the proposal's team, or default platform-eng.
2. **Lookup budget** — call get_team_budget with that team.
3. **Compute remaining** — remaining_usd = monthly_budget_usd – spent_usd.
4. **Decide** — **Approve** if projected_monthly_usd ≤ remaining_usd; **Reject** if greater.
5. **Record reason** — include team, budget, spend, remaining, and projected cost in the verdict.

Worked demo example

Input	Value / outcome
Team	platform-eng

Input	Value / outcome
Budget / spent / remaining	\$500 / \$47 / \$453
Proposed 8xA100 (~\$12,400/mo)	Reject — exceeds remaining
Proposed small CPU (~\$40/mo)	Approve — within remaining

5. Policy rules (agents must enforce)

1. **Hard cap** — do not approve if `spent_usd + projected_monthly_usd` would exceed `monthly_budget_usd`.
2. **Default team** — if team is missing, use `platform-eng` (\$500/mo).
3. **Fresh lookup** — always call `get_team_budget` at decision time.
4. **Fail closed on overspend** — reject when remaining capacity is insufficient.
5. **Field-name contract** — snake_case only: `team`, `monthly_budget_usd`, `spent_usd`.

6. Optional: orphaned / idle deployment policy

If the ToolSet can surface idle resources, agents may reclaim budget. Idle > 7 days with no traffic → flag orphaned; recommend tear-down. Optional tool: `list_orphaned_deployments` returning `deployment_id`, `team`, `monthly_cost_usd`, `idle_days`.
Optional for hackathon — `get_team_budget` alone is sufficient for Guardian.

7. ToolSet naming & export checklist (Nexla MCP Studio)

Step	Instruction
Upload	This PDF into Nexla MCP Studio
ToolSet name	SecGate-Budget-Governance
Verify tools	At least <code>get_team_budget</code> ; ideally also <code>list_team_budgets</code>
Seed data	One row: <code>platform-eng / 500 / 47</code>
Export as MCP	URL like <code>https://api-genai.nexla.io/mcp/service_key/<server_key></code>
Auth	Org Service Key as Bearer with the MCP URL

SecGate .env after export

```
NEXLA_USE_SHIM=0
NEXLA_MCP_URL=https://api-genai.nexla.io/mcp/service_key/<server_key>
NEXLA_SERVICE_KEY=<existing service key already in .env>
NEXLA_BUDGET_TOOL=get_team_budget
NEXLA_TEAM=platform-eng
```

8. Success criteria

- MCP tool `get_team_budget` returns { `team`, `monthly_budget_usd`, `spent_usd` }.
- Demo team `platform-eng` shows budget 500 and spend ~47.
- Guardian approves small costs and rejects oversized GPU proposals against remaining budget.
- Control Tower budget badge shows **Nexla** when real MCP URL is set (`NEXLA_USE_SHIM=0`).